

Hybrid Context Management

Pinned facts + a rolling summary of the distant past + a verbatim recent window

The idea in one sentence. Instead of choosing one way to shrink a conversation, keep three stores side by side — each holding a different *kind* of information, each shrunk by a different rule. This is the configuration most production agents converge on.

1. Why one strategy is never enough

A long conversation will not fit in the context window. There are three obvious fixes, and each one breaks in its own way:

Strategy	What it does	Where it breaks
Send everything	Resend the whole transcript each turn.	Cost grows quadratically, and eventually it simply does not fit.
Sliding window	Keep only the last N messages; drop the rest.	Cheap and perfectly accurate — about the recent past. It forgets the user's name, because that was said at turn 1.
Summarisation	Compress old turns into a recap.	Remembers the gist. But re-compress a summary enough times and exact details drift — a name gets dropped, or who-said-what inverts.

The insight. These are not three competing answers to one question. They are three *right* answers to three *different* questions. A user's name and a rambling three-turn digression do not deserve the same treatment — so stop giving it to them.

2. The three layers

Pinned facts

Must be exact.

Names, IDs, file paths, deadlines, hard constraints. Small, and copied character-for-character.

Never summarised.

Rolling summary

Must be cheap.

The gist of the distant past: what was decided, what was ruled out, what topics were covered.

Never allowed to grow.

Recent window

Must be complete.

The last few turns, word for word — this is what "it", "that one" and "the second option" refer to.

Never compressed.

Each layer earns its place by answering a need the other two cannot. Pinned facts buy *precision*. The summary buys *reach*. The window buys *fidelity*. Drop any one and you get back one of the failure modes from section 1.

3. What every request looks like

The three stores are concatenated in a fixed order, oldest-and-most-durable first, then this turn's question. The model sees one ordinary message list:

1 · PINNED FACTS

~80 tokens · hard cap

[Known facts about this user and task]

name: Sourav

language: Python

book recommended: "Designing Data-Intensive Applications" (Kleppmann)

chosen consensus algorithm: Raft (easier to implement than Paxos)

Rewritten only when a fact changes. Never paraphrased.

2 · ROLLING SUMMARY

~200 tokens · hard cap

[Summary of earlier conversation]

Compared Paxos and Raft; Raft preferred for clarity. Covered the CAP theorem with DynamoDB as an AP example and Spanner as CP. Discussed leader election and behaviour during a network partition.

Re-folded on each compaction, so it never grows past its cap.

3 · RECENT WINDOW (VERBATIM)

~600 tokens · the flexible layer

user: What's the role of the term number in Raft?

assistant: A term is a logical clock. Each election increments it, and...

user: How is that different from Paxos's ballot numbers?

assistant: Ballot numbers serve a similar purpose, but...

Untouched, byte for byte. The last N messages, always complete.

4 · THIS TURN'S QUESTION

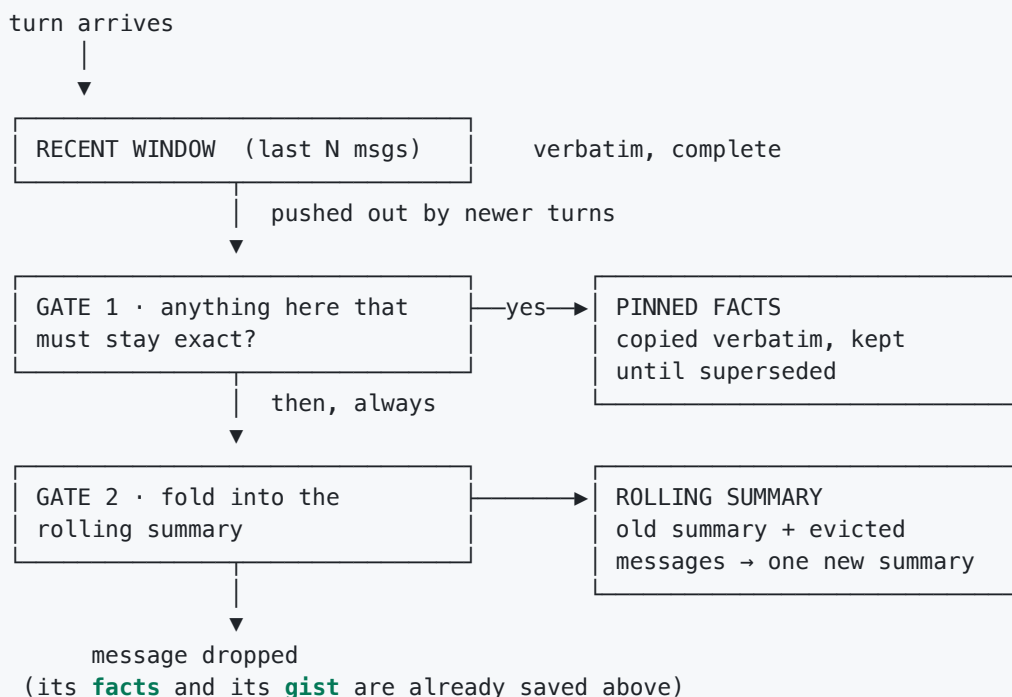
appended last

user: By the way, what was my name again?

The current question is appended *after* any compaction, so it always reaches the model word for word.

4. How it actually works: the life of one message

Nothing is ever deleted outright. A message ages out of the verbatim window, but on its way out it must pass two gates — and each gate may keep a piece of it:



That ordering is the whole trick. **Gate 1 runs before Gate 2**, so anything that has to be exact is lifted out *before* the lossy step ever sees it. A name never depends on the summariser choosing to keep it.

When does any of this run?

Only when the next request would not fit. Before each API call:

```
candidate = pinned + summary + window + [new question]

if count_tokens(candidate) > BUDGET:
    evicted = oldest messages from the window
    pinned += extract_facts(evicted)           # Gate 1 – exact
    summary = summarise(summary + evicted)     # Gate 2 – lossy
    window = window without evicted
```

Checking *before* the call is what makes the budget a real ceiling: no request ever goes out over budget. Checking afterwards would let the turn that trips the limit ship oversized.

5. A worked example

One conversation, watched at three moments. Watch the three layers move.

Turn 2 — everything still fits

PINNED

9 tok

name: Sourav

SUMMARY

0 tok

(empty – nothing is distant yet)

WINDOW

4 messages, verbatim

user: Hi, my name is Sourav.
assistant: Hi Sourav! How can I help?
user: What's a good book on distributed systems?
assistant: "Designing Data-Intensive Applications" by Martin Kleppmann...

The name was pinned the moment it was stated — not later, when it was about to be lost.

Turn 12 — the budget is hit for the first time

Turns 1–4 are pushed out of the window. Gate 1 lifts the book title out as an exact fact; Gate 2 folds the rest into a first summary. Then the messages are dropped.

PINNED

31 tok · grew by 2 facts

name: Sourav
language: Python
book recommended: "Designing Data-Intensive Applications" (Kleppmann)

SUMMARY

44 tok · created

Asked for a distributed-systems book and why it was chosen; noted a preference for Python examples.

WINDOW

8 messages, verbatim

...turns 5–12, untouched...

Turn 27 — the probe that separates hybrid from the alternatives

The user asks: "By the way, what was my name again?" Turn 1 was dropped fifteen turns ago and the summary has been re-folded four times. It does not matter:

PINNED38 tok · still exact

name: Sourav ← answered straight from here
language: Python
book recommended: "Designing Data-Intensive Applications" (Kleppmann)
chosen consensus algorithm: Raft

SUMMARY198 tok · at its cap

Covered Paxos vs Raft, the CAP theorem, leader election, partitions, log replication and Kafka's ISR...

WINDOW8 messages, verbatim

...turns 23–27, untouched...

Why it worked.

"Sourav" was never handed to the summariser. It was copied once, exactly, and has sat in the pinned block ever since — costing about 4 tokens a turn.

6. The budget: who gets what

Give each layer its own ceiling, and let the window absorb whatever is left. With a 1,000-token budget:

Layer	Budget	Behaviour when full
Pinned facts	120 tok (hard)	Drop the least-recently-referenced fact. If this fills up often, the extractor is pinning things that are not really facts.
Rolling summary	250 tok (hard)	Re-fold: old summary + newly evicted messages → one new summary. Never a chain of summaries.
Recent window	~600 tok (flexes)	Evict the oldest pair through the two gates until the request fits.

Only the window flexes. That keeps the prompt prefix — pinned plus summary — effectively constant, so cost per turn is flat instead of creeping upward.

7. Why it beats each strategy alone

Question at turn 27	Sliding window	Summarisation	Hybrid
"What was my name again?"	✗ evicted	~ usually, may drift	✓ pinned, exact
"What did we decide about Raft?"	✗ out of window	✓ in the summary	✓ in the summary
"Expand on your last point."	✓ verbatim	✓ verbatim	✓ verbatim
Cost curve	flat	sawtooth	flat, small sawtooth

8. What it costs you

- **An extra call per eviction — sometimes two.** Summarising costs one call; fact extraction may cost another. Budget for it, and measure it: report what compaction spent next to what it saved.
- **Facts go stale.** "We're using Postgres" can be superseded by "actually, let's use SQLite." Pinned facts must be *updatable by key*, not append-only — otherwise the block fills with contradictions and the model picks one at random.
- **The extractor can over-pin.** If everything looks like a fact, the pinned block becomes a second uncapped transcript. A hard cap plus a strict prompt is the defence.
- **Three moving parts, three ways to be wrong.** Log which layer answered each probe. When recall fails you need to know whether the fact was never pinned, was pinned then evicted, or was pinned and ignored.

Rule of thumb. If losing a detail would make the answer *wrong*, pin it. If losing it would only make the answer *vaguer*, let the summary carry it. If the next question probably refers to it directly, it belongs in the verbatim window.

Hybrid context management · companion note to `agent_context_management_sliding_window.py` and `agent_context_management_summerisation.py`